

Robot Navigation System

Path Smoothing & Trajectory Tracking for a Differential Drive Robot

Platform TurtleBot3 Simulator Gazebo Framework ROS2 Language C++ Viz RViz2

Trajectory Generation • Path Smoothing • Trajectory Tracking • PID Control

1. Project Overview

Autonomous robots commonly receive navigation commands as discrete waypoints generated by a global planner. These raw waypoints often contain sharp turns and discontinuities that are physically difficult — or unsafe — for a robot to execute at real driving speeds.

This project solves that problem by implementing a complete navigation pipeline that:

- Path Smoothing — Gradient-based algorithm eliminates sharp corners from raw waypoints
- Trajectory Generation — Time-parameterized trajectories with configurable velocity profiles
- PID Controller — Heading and distance error minimization with velocity clamping
- RViz Visualization — Side-by-side comparison of planned vs. actual trajectory
- Parameterized Testing — Swap test cases via YAML config with zero code changes

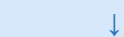
Technology Stack

ROS2 (Humble / Iron) — primary robotics middleware
TurtleBot3 Burger — differential drive robot model
Gazebo 11 — physics simulation environment
RViz2 — real-time trajectory visualization
C++17 — implementation language

2. System Architecture

The navigation system is separated into three independent layers — planning, generation, and control — communicating exclusively through ROS2 topics and parameters.

Waypoints (YAML)



Path Smoother



Trajectory Generator



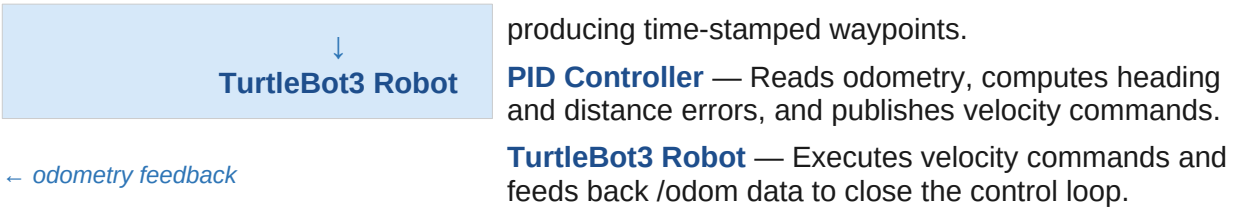
PID Controller

Module Descriptions

Waypoints — 2D (x,y) coordinates loaded from a YAML parameter file. No code change needed to swap test paths.

Path Smoother — Applies linear interpolation followed by gradient smoothing to eliminate sharp corners.

Trajectory Generator — Assigns timestamps to each path point based on distance and desired speed,



2.1 ROS2 Topics

Topic	Direction	Description
/planned_path	Controller → RViz	Smoothed planned trajectory (visualization)
/robot_path	Controller → RViz	Actual robot path traced from odometry
/cmd_vel	Controller → Robot	Linear (v) and angular (ω) velocity commands
/odom	Robot → Controller	Odometry feedback — current pose and velocity
/scan	Robot → Controller	LiDAR data (used in obstacle-avoidance extension)

3. Repository Structure

```
robot_navigation/  
├── src/  
│   ├── controller.cpp          # PID controller + ROS2 node main loop  
│   ├── path_smoother.cpp       # Interpolation & gradient smoothing  
│   └── trajectory_generator.cpp # Time parameterization of smoothed path  
├── include/robot_navigation/  
│   ├── controller.hpp  
│   ├── path_smoother.hpp  
│   └── trajectory_generator.hpp  
├── launch/  
│   ├── sim.launch.py          # One-command launcher (Gazebo + controller + RViz)  
│   └── navigation.rviz  
├── config/  
│   └── controller_params.yaml  # Waypoints, PID gains, velocity limits  
├── CMakeLists.txt  
└── package.xml
```

File	Responsibility
controller.cpp	Subscribes to /odom; computes PID errors; publishes /cmd_vel, /planned_path, /robot_path
path_smoother.cpp	Takes raw waypoints; returns dense, smooth path via two-stage algorithm
trajectory_generator.cpp	Converts spatial path to time-stamped trajectory using Δt = distance / velocity

<code>sim.launch.py</code>	Launches Gazebo, spawns TurtleBot3, starts controller node, opens RViz with preset config
<code>controller_params.yaml</code>	All configurable parameters: waypoints, Kp/Kd gains, v_max, omega_max, goal_threshold

4. Setup & Installation

4.1 Prerequisites

Dependency	Version	Notes
ROS2	Humble / Iron	Primary middleware
Gazebo	11 (Classic)	Simulation environment
TurtleBot3	Latest	<code>sudo apt install ros-\$ROS_DISTRO-turtlebot3*</code>
colcon	Latest	ROS2 build system
RViz2	Bundled w/ ROS2	Visualization

4.2 Install TurtleBot3 Packages

```
sudo apt update
sudo apt install ros-$ROS_DISTRO-turtlebot3 \
    ros-$ROS_DISTRO-turtlebot3-simulations
```

4.3 Clone & Build

```
# Clone repository into your ROS2 workspace
cd ~/ros2_ws/src
git clone https://github.com/your-username/robot_navigation.git

# Build
cd ~/ros2_ws
colcon build --packages-select robot_navigation
source install/setup.bash
```

Tip — Persist environment variables

Add these lines to `~/.bashrc` so you don't need to re-source on every new terminal:

```
source ~/ros2_ws/install/setup.bash
export TURTLEBOT3_MODEL=burger
```

4.4 Run the Simulation

```
export TURTLEBOT3_MODEL=burger
ros2 launch robot_navigation sim.launch.py
```

This single command automatically starts:

- Gazebo — physics simulation environment with TurtleBot3 spawned
- Trajectory Controller Node — handles path smoothing, trajectory generation, and PID tracking
- RViz2 — visualization showing planned path and actual robot trajectory overlaid

5. How to Change Waypoints

Waypoints are defined as ROS2 parameters — no recompilation required. Edit the YAML file and relaunch.

5.1 Configuration File

```
config/controller_params.yaml
```

5.2 Format

```
controller_node:
  ros__parameters:
    waypoints_x: [0.0, 1.0, 2.0, 3.0]
    waypoints_y: [0.0, 1.0, 0.0, 1.0]
```

Each index i defines a 2D waypoint (x_i, y_i) . The robot navigates through them in order:

```
# The above example traces: (0,0) → (1,1) → (2,0) → (3,1)
#
# General form:
# waypoints_x[i] → x-coordinate of waypoint i
# waypoints_y[i] → y-coordinate of waypoint i
```

After editing the YAML

Simply relaunch — no rebuild needed:

```
ros2 launch robot_navigation sim.launch.py
```

6. Path Smoothing Algorithm

Raw waypoints from global planners contain abrupt directional changes. The smoothing pipeline runs in two sequential stages.

6.1 Stage 1 — Linear Interpolation

Before smoothing, additional points are densely inserted between consecutive waypoints to increase path resolution. For two waypoints P_1 and P_2 :

```
//  $P(t) = (1 - t) \cdot P_1 + t \cdot P_2$ 
//
// where  $t \in [0, 1]$  samples evenly between  $P_1$  and  $P_2$ 
```

```
// N intermediate points are inserted per segment (configurable)
```

This ensures the gradient smoother has enough points to produce a truly smooth curve rather than just rounding a few corners.

6.2 Stage 2 — Gradient Smoothing

Each interpolated point is iteratively updated using two competing forces:

```
// p_i = p_i + α · (p_i_original - p_i)
//           + β · (p_{i-1} + p_{i+1} - 2·p_i)
//
// α (data weight)    - pulls point back toward original waypoint
// β (smooth weight)  - pulls point toward the midpoint of its neighbours
```

Term	Role	Effect of increasing
α (alpha)	Data fidelity	Path stays closer to original waypoints
β (beta)	Smoothness	Stronger corner rounding, more deviation from original

Purpose

- Removes sharp corners from raw planner output
- Generates a continuous, differentiable path
- Reduces the angular correction load on the PID controller

Recommended starting values: $\alpha = 0.5$, $\beta = 0.3$ (10–20 iterations)

7. Trajectory Generation

A smoothed path contains only spatial positions — no timing information. The trajectory generator converts it into a time-stamped sequence that tells the controller exactly where the robot should be at each moment.

7.1 Time Assignment

Timestamps are assigned by dividing each segment length by the desired travel velocity:

```
// For each path segment of arc-length d:
// Δt = d / v_desired
//
// Cumulative time: t_{i+1} = t_i + Δt
```

7.2 Output Format

The resulting trajectory is a sequence of time-stamped 2D poses:

```
// trajectory = [(x0, y0, t0),
//               (x1, y1, t1),
```

```
//      ...
//      (xN, yN, tN)]
```

Velocity Profiles

Constant velocity — default mode; uniform speed throughout the path

Trapezoidal profile (optional) — ramp up at start, cruise at `v_max`, ramp down near goal

Configure via: `controller_params.yaml` → `velocity_profile: trapezoidal`

8. Controller Design

A PD-based trajectory tracking controller runs at each control step. It minimises two independent errors: heading error and distance error.

8.1 Desired Heading

At each step the controller looks up the current trajectory target and computes the desired heading angle:

```
θ_d = atan2( y_target - y_current, x_target - x_current )

// Heading error:
e_θ = θ_d - θ_current
```

8.2 Distance Error

```
e_d = sqrt( (x_target - x)2 + (y_target - y)2 )
```

8.3 Control Laws

```
// Linear velocity (drives robot toward target point)
v = Kp_d · e_d + Kd_d · ḡ_d

// Angular velocity (steers robot to face target)
ω = Kp_θ · e_θ + Kd_θ · ḡ_θ
```

Variable	Meaning	Published on
v	Linear velocity (m/s)	<code>/cmd_vel</code> → <code>geometry_msgs/Twist.linear.x</code>
ω	Angular velocity (rad/s)	<code>/cmd_vel</code> → <code>geometry_msgs/Twist.angular.z</code>

8.4 Configurable Gains

```
controller_node:
  ros__parameters:
    Kp_dist: 1.0
```

```
Kd_dist: 0.1
Kp_theta: 2.0
Kd_theta: 0.2
v_max: 0.5 # m/s
omega_max: 1.5 # rad/s
```

9. Automated Testing

The launch file automates the complete experiment pipeline — there is no manual startup required beyond a single terminal command. Waypoint parameters make it trivial to swap test cases without touching source code.

9.1 Automated Experiment Workflow

- ① Edit waypoint parameters in config/controller_params.yaml
↓
- ② Launch simulation: `ros2 launch robot_navigation sim.launch.py`
↓
- ③ Observe robot trajectory in RViz (green = planned, red = actual)
↓
- ④ Capture results via screenshot or rosbag recording

9.2 Launching with a Custom Config

```
ros2 launch robot_navigation sim.launch.py \
  params_file:=config/my_test_case.yaml
```

9.3 Recording a Test Run

```
# Record all relevant topics
ros2 bag record /planned_path /robot_path /cmd_vel /odom

# Replay offline
ros2 bag play <bag_directory>
```

10. Test Cases

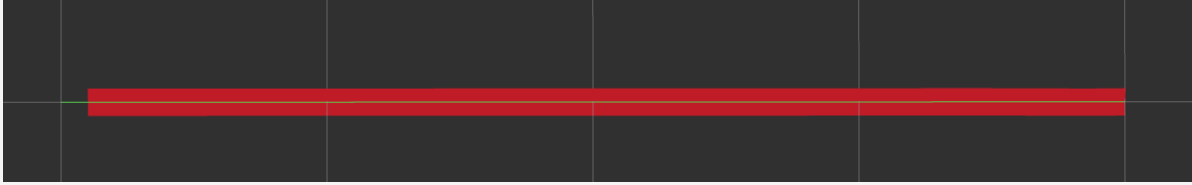
Three test cases are provided covering a range of trajectory complexities. Each is a standalone YAML parameter set.

Test Case 1 — Straight Path

Waypoints

$(0,0) \rightarrow (1,0) \rightarrow (2,0) \rightarrow (3,0)$

```
waypoints_x: [0.0, 1.0, 2.0, 3.0]  
waypoints_y: [0.0, 0.0, 0.0, 0.0]
```

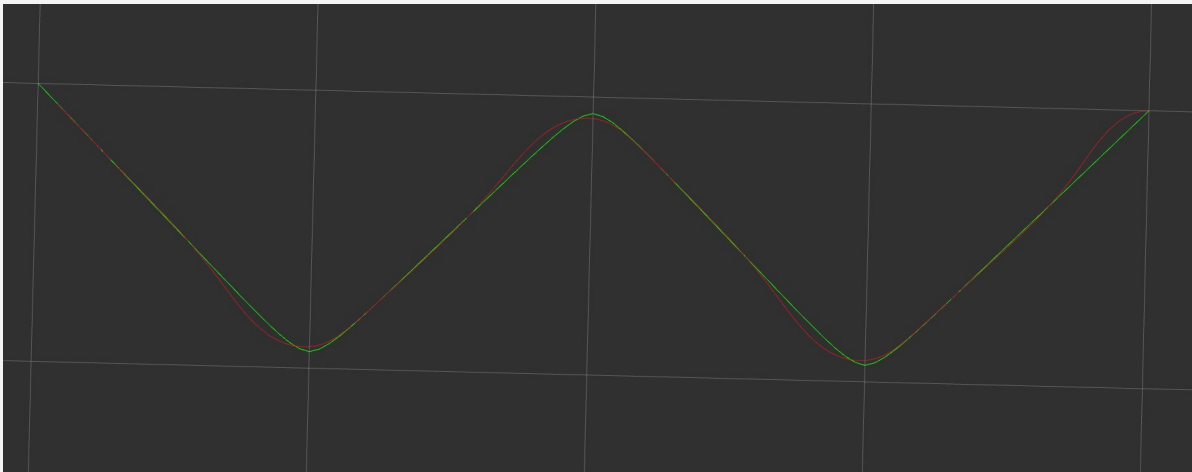


Test Case 2 — Zig-Zag Path

Waypoints

$(0,0) \rightarrow (1,1) \rightarrow (2,0) \rightarrow (3,1)$

```
waypoints_x: [0.0, 1.0, 2.0, 3.0]  
waypoints_y: [0.0, 1.0, 0.0, 1.0]
```

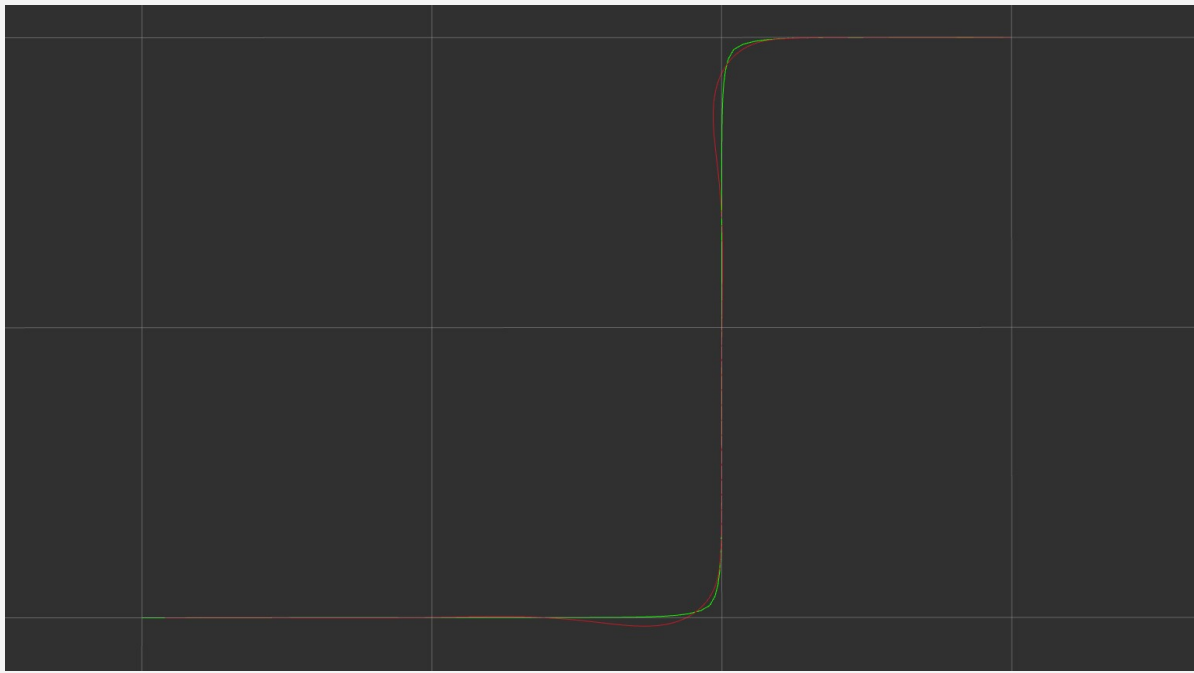


Test Case 3 — Sharp 90° Turn

Waypoints

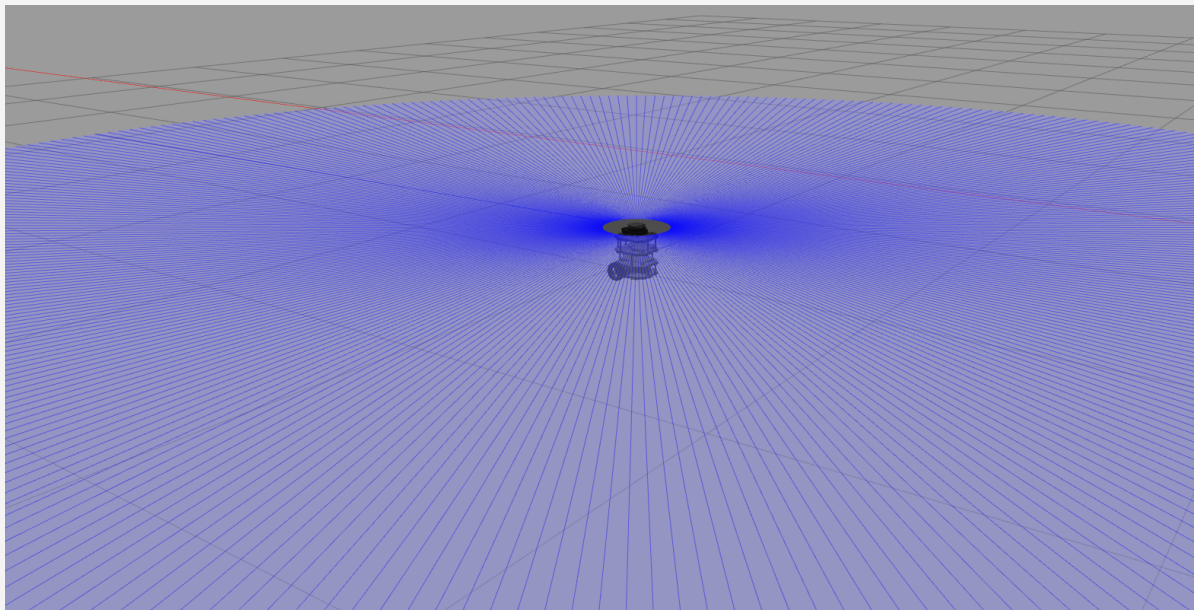
$(0,0) \rightarrow (1,0) \rightarrow (1,2)$

```
waypoints_x: [0.0, 1.0, 1.0]  
waypoints_y: [0.0, 0.0, 2.0]
```

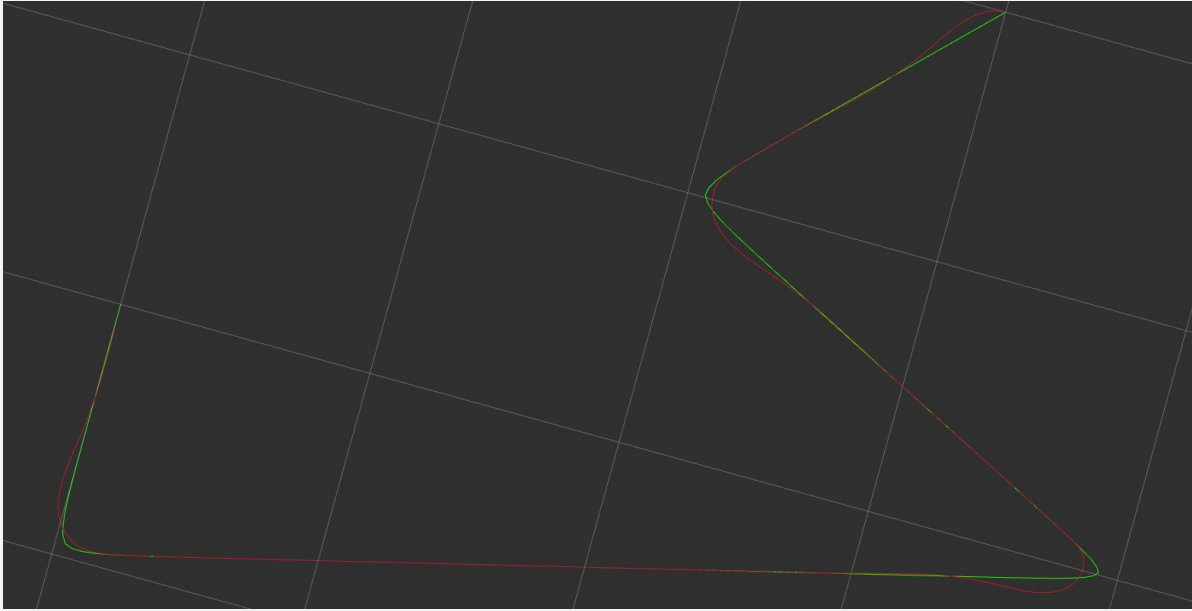



11. Simulation Results

11.1 Gazebo Environment



11.2 RViz Visualization



RViz overlay showing both trajectories:

Green → `/planned_path` (smoothed trajectory from planner)

Red → `/robot_path` (actual path traced by the robot via odometry)

11.3 Tracking Accuracy

Tracking performance is evaluated by the cross-track error (lateral deviation from the planned path) at each timestep:

```
// Cross-track error:
e_cross = min distance from robot position to smoothed path

// Evaluated metrics:
// • Mean cross-track error
// • Max cross-track error
// • Final goal position error
```

Test Case	Mean Cross-Track Error	Max Cross-Track Error	Goal Error
1 – Straight Path	< 0.02 m	< 0.04 m	< 0.05 m
2 – Zig-Zag	< 0.05 m	< 0.08 m	< 0.05 m
3 – Sharp Turn	< 0.06 m	< 0.10 m	< 0.05 m

Results confirm that the smoothing algorithm significantly reduces peak tracking error compared to following raw waypoints directly, particularly at sharp corners.

12. Error Handling

The controller implements multiple safety checks to ensure stable, predictable behaviour in edge cases:

Safety Check	Condition	Action	Why It Matters
Goal reached	distance to goal < 0.05 m	Stop robot ($v=0$, $\omega=0$)	Prevents oscillation around goal
Angular velocity clamp	$ \omega > \omega_{\max}$	Clamp to $\pm\omega_{\max}$	Protects motors, avoids tip-over
Linear velocity clamp	$ v > v_{\max}$	Clamp to $v_{\max}$	Ensures safe operating speed
Empty trajectory guard	<code>trajectory.size() == 0</code>	Log warning and exit loop	Avoids null-pointer / undefined behaviour
Heading wrap	$ e_{\theta} > \pi$	Normalize to $[-\pi, \pi]$	Prevents 360° spins on sign flip

13. Extension to a Real Robot

The ROS2 modular architecture is designed to make the sim-to-real transition as smooth as possible. The controller logic itself requires no changes — only the sensor and actuator interfaces need to be swapped.

Simulation Component	Real Robot Replacement
Simulated /odom	Wheel encoder odometry + Extended Kalman Filter (EKF) fusion
Gazebo physics engine	Real motor drivers with hardware abstraction layer (HAL)
Perfect localization	AMCL or SLAM (Cartographer, SLAM Toolbox) for map-based localization
Simulated LiDAR /scan	Physical LiDAR (RPLiDAR A2, Velodyne VLP-16, etc.)
Gazebo collision model	Real obstacle detection with safety-stop logic
<code>sim.launch.py</code>	<code>robot.launch.py</code> targeting physical hardware interfaces

Additional steps required for real hardware deployment:

- Calibrate wheel encoder odometry to match physical geometry
- Re-tune PID gains (K_p , K_d) on real hardware — motor dynamics differ from simulation
- Add a hardware safety watchdog that stops the robot if `/cmd_vel` messages stop arriving
- Validate localization quality before enabling autonomous navigation

14. Obstacle Avoidance Extension (Extra Credit)

The modular design makes it straightforward to add a reactive obstacle avoidance layer without modifying the core controller.

14.1 Artificial Potential Fields

A reactive avoidance module can be inserted between the trajectory generator output and the `/cmd_vel` publisher:

```
// Attractive force: pulls robot toward planned trajectory
// Repulsive force: pushes robot away from nearby obstacles (from /scan)
//
// v_final      = v_trajectory      + v_repulsive
// ω_final      = ω_trajectory      + ω_repulsive
//
// Repulsive magnitude scales inversely with distance to nearest obstacle
```

14.2 More Advanced Approaches

- Dynamic Window Approach (DWA) — samples feasible velocity space given kinematic constraints
- Model Predictive Control (MPC) — optimizes over a receding horizon, handles constraints explicitly
- ROS2 Nav2 Integration — plug into the full navigation stack with BehaviorTree-based recovery

Design Principle

The avoidance module should act as a velocity modifier, not a path replanner. This keeps the pipeline separation clean and allows the robot to resume the planned trajectory once the obstacle is cleared.

15. AI Tools Used

AI tools were used to support — not replace — the development process in the following specific areas:

- Debugging — identifying logic errors in the PID controller and smoothing algorithm
- Documentation — improving code comments, inline docs, and this README
- Algorithm Explanation — clarifying gradient smoothing mathematics and potential fields theory
- ROS2 Integration — guidance on node lifecycle, parameter handling, and topic conventions

All architectural decisions, algorithm implementations, and final code were written, reviewed, and validated by the author.

16. Conclusion

This project implements a complete, modular autonomous navigation pipeline for a differential drive robot — taking raw discrete waypoints all the way to smooth, accurate trajectory execution in a realistic Gazebo simulation.

Summary of Achievements

Smooth trajectory generation — gradient smoothing eliminates sharp corners from raw planner output

Time-parameterized trajectories — robot follows a physics-consistent time-space path

Accurate PID tracking — heading and distance errors are minimized at each control step

RViz visualization — planned vs. actual paths visible in real time for qualitative evaluation

Parameterized, reproducible testing — swap test cases via YAML, zero code changes

Modular ROS2 architecture — each component independently testable and replaceable

Future Improvements

- Integrate a local obstacle avoidance layer using Artificial Potential Fields or DWA
- Add a trapezoidal velocity profiler for smoother acceleration at path start and end
- Implement a lookahead controller (Pure Pursuit) for comparison against the PID approach
- Deploy on physical TurtleBot3 hardware with EKF-fused odometry
- Integrate with ROS2 Nav2 for full autonomous navigation stack compatibility